

C++ Project / Planning

Trivia Quiz System

Student 1	Ion Popescu	Student 2	Maria Ionescu
Project	Trivia Quiz System	Language	C++17

I. Task Description

Application 1 (main) — Player App — Student 1: Ion Popescu

- Play the trivia quiz by submitting answers via command-line arguments
- Save mid-game progress to progress.txt and resume across separate runs
- Save final scores to leaderboard.txt when the quiz is completed
- View the leaderboard sorted by score (descending)
- Reset saved progress on demand

Application 2 (admin) — Admin App — Student 2: Maria Ionescu

- Add new trivia questions to questions.txt (immediately visible to App 1)
- Delete individual questions by number
- View and delete players from the leaderboard written by App 1
- Clear the entire leaderboard
- View and reset saved player progress written by App 1

Bidirectional flow: App 1 writes leaderboard.txt and progress.txt; App 2 reads them. App 2 writes questions.txt; App 1 reads it.

II. Data Structures (Classes)

Class: Question

Stores a trivia question with four options, the correct answer index, and difficulty. Overloads **operator<<** for stream display.

text	string	The question text shown to the player
options[4]	string	The four answer choices
correct	int	Index of correct answer (1–4)
difficulty	int	1=Easy, 2=Medium, 3=Hard
difficultyLabel()	string	Returns 'Easy', 'Medium', or 'Hard'
operator<<	friend ostream&	Overloaded stream output — enables cout << question

Class: Player (base class)

Holds a player's name and score. Base class for GamePlayer. Provides virtual saveToLeaderboard() and overloads **operator<<**.

name	string	Player's display name
score	int	Current accumulated score
getName() / getScore()	accessors	Return name and score
setScore() / incrementScore()	void	Modify score value
saveToLeaderboard()	virtual void	Appends name and score to leaderboard.txt
operator<<	friend ostream&	Overloaded stream output — enables cout << player

Class: GamePlayer (inherits Player)

IS-A Player (public inheritance). **HAS-A vector<Question>** (association — used during play). Adds stateless progress persistence across program invocations. Managed via **unique_ptr<GamePlayer>** in main.

questionIndex	int	0-based index of next unanswered question
getQuestionIndex()	int	Returns the current question position
advanceQuestion()	void	Moves to the next question
saveProgress()	void	Writes name, index, score to progress.txt
loadProgress()	bool	Reads progress.txt; returns true if matched
clearProgress()	static void	Writes 'none 0 0' sentinel to progress.txt

Class: User (base class) — admin.cpp

Represents a generic system user. Base class for Admin. Declares virtual getRole() so derived classes can identify their role.

name	string	User's display name
getName()	string	Returns the user's name
getRole()	virtual string	Returns 'User' (overridden in Admin)

Class: Admin (inherits User) — admin.cpp

IS-A User (public inheritance). Overrides `getRole()` and adds password authentication. Managed via `unique_ptr<Admin>` in main.

<code>authenticate(pwd)</code>	<code>bool</code>	Returns true if password matches 'admin123'
<code>getRole()</code>	<code>string override</code>	Returns 'Admin'

Template Function: `displayList`

Generic template function used in both apps. Prints a numbered list of any type `T` that supports operator<<. Works for both `vector<Question>` and `vector<Player>` without code duplication.

```
template <typename T>
void displayList(const vector<T>& items, const string& header)
```

III. File Structure

`questions.txt`

Written by: App 2 (admin) **Read by:** App 1 (main)

Stores all trivia questions. Admin writes it; player app reads it.

Format:

```
<question text>
<option 1> <option 2> <option 3> <option 4> (one per line)
<correct answer index 1-4>
<difficulty 1-3>
[block repeats for each question]
```

`leaderboard.txt`

Written by: App 1 (main) **Read by:** App 2 (admin)

Stores completed game scores. Player app appends; admin reads and manages.

Format:

```
<player_name> <score>
[one line per completed game]
```

`progress.txt`

Written by: App 1 (main) **Read by:** App 2 (admin)

Stores active mid-game state. Sentinel 'none 0 0' means no active session.

Format:

```
<player_name> <question_index> <score>
[single line – 'none 0 0' when empty]
```

IV. Commands the Apps Will Implement

Application 1 — main (Player App)

--	--

<code>./main play <name></code>	Start a new game or resume saved progress for the named player
<code>./main answer <name> <1-4></code>	Submit an answer to the current question
<code>./main answer <name> 0</code>	Save current progress and exit without answering
<code>./main leaderboard</code>	Display all scores from leaderboard.txt sorted by score descending
<code>./main questions</code>	Display all questions from questions.txt
<code>./main progress</code>	Show current saved progress (player, question, score)
<code>./main reset <name></code>	Erase saved progress for the named player

Application 2 — admin (Admin App)

<code>./admin view_questions</code>	Display all questions in questions.txt with correct answers
<code>./admin add_question <text> <o1> <o2> <o3> <o4> <correct> <diff></code>	Append a new question — immediately visible to App 1
<code>./admin delete_question <number></code>	Remove a question by its display number
<code>./admin view_leaderboard</code>	Read and display leaderboard.txt written by App 1
<code>./admin delete_player <name></code>	Remove all entries for a player from the leaderboard
<code>./admin clear_leaderboard</code>	Empty leaderboard.txt completely
<code>./admin view_progress</code>	Read and display progress.txt written by App 1
<code>./admin reset_progress</code>	Reset progress.txt to the 'none 0 0' empty state